PULSO

ENTREGA 1 / E6

Base técnica del repositorio

Proyecto Final Backend · 13 de noviembre de 2026

Miguel Ángel Márquez Coronel · María Gabriela Rey Barrios · Andrés Felipe Serrano Mendoza

8. E6. Base técnica del repositorio y estado de implementación

8.1. Descripción general

La base técnica de PULSO está constituida por una arquitectura modular que integra una aplicación web, una API backend, un servicio independiente de Machine Learning, una base de datos relacional y un sistema RAG para la generación de recomendaciones educativas.

La solución utiliza React y TypeScript para la interfaz, Python y FastAPI para los servicios backend, XGBoost y SHAP para las operaciones predictivas y explicativas, PostgreSQL para cuentas e historiales de análisis, y n8n junto con Supabase para las recomendaciones educativas fundamentadas documentalmente.

El backend sigue una adaptación de Clean Architecture, con separación entre dominio, aplicación, infraestructura e interfaces HTTP.

La implementación incorpora mecanismos de autenticación mediante JWT, protección de contraseñas con Argon2 y persistencia de resultados de análisis asociados con usuarios registrados.

8.2. Organización del repositorio

PULSO está organizado en un repositorio que agrupa los componentes de aplicación, servicios, infraestructura, pruebas y documentación.

Directorio o archivo	Responsabilidad
frontend/	Interfaz web React, TypeScript y Vite
backend/	API FastAPI, autenticación y casos de uso
backend/app/domain/	Entidades, reglas, errores y puertos
backend/app/application/use_cases/	Casos de uso de análisis, usuarios e historial
backend/app/infrastructure/	Adaptadores, seguridad, configuración y repositorios
backend/app/interfaces/http/	Rutas, esquemas e inyección de dependencias
backend/migrations/	Migraciones SQL de PostgreSQL
backend/tests/	Pruebas automatizadas
ml_service/	Servicio de Machine Learning
ml/	Preparación, entrenamiento y evaluación del modelo
artifacts/	Artefactos del modelo entrenado
infrastructure/nginx/	Configuración del proxy inverso
.github/workflows/	Workflows de integración continua
docs/	Documentación técnica
docker-compose.yml	Definición de servicios contenerizados

La estructura permite separar las responsabilidades de presentación, lógica de aplicación, persistencia e inferencia predictiva.

8.3. Backend y API REST

El backend está desarrollado en Python mediante FastAPI.

Su función es coordinar las operaciones de análisis cardiovascular, autenticación, consulta de historiales y generación de recomendaciones educativas.

La API expone los siguientes endpoints principales:

Método	Endpoint	Responsabilidad	Autenticación
GET	/api/health	Comprobar disponibilidad	No
POST	/api/auth/register	Registrar usuarios	No
POST	/api/auth/login	Iniciar sesión	No
GET	/api/auth/me	Consultar usuario	Sí

Método	Endpoint	Responsabilidad	Autenticación
		autenticado	
POST	/api/analyze	Predicción y explicación SHAP	Sí
POST	/api/rag	Recomendaciones educativas	Sí
POST	/api/simulate	Simulación de escenarios	Sí
GET	/api/analyses	Consultar historial de análisis	Sí

Las solicitudes y respuestas se validan mediante esquemas Pydantic.

Los endpoints protegidos utilizan autenticación HTTP Bearer y validación del token JWT.

El backend transforma los errores de dominio, validación, autenticación y comunicación externa en respuestas HTTP estructuradas.

8.4. Gestión de usuarios y autenticación

PULSO incorpora funcionalidades de registro, inicio de sesión y consulta de información del usuario autenticado.

Estas operaciones se organizan mediante casos de uso y adaptadores de infraestructura que respetan la separación de responsabilidades del backend.

8.4.1. Registro de usuarios

El caso de uso RegisterUser permite registrar una cuenta mediante correo electrónico y contraseña.

La implementación utiliza el repositorio de usuarios para conservar las credenciales protegidas en PostgreSQL y gestionar la unicidad de los correos electrónicos.

8.4.2. Inicio de sesión

El caso de uso AuthenticateUser valida las credenciales proporcionadas.

Cuando la autenticación es satisfactoria, el sistema emite un token de acceso que permite utilizar las operaciones protegidas.

Las credenciales incorrectas generan una respuesta de autenticación fallida.

8.4.3. Protección de contraseñas

La implementación utiliza Argon2 mediante pwdlib.

El puerto PasswordHasher permite desacoplar el proceso de protección de contraseñas de su implementación concreta, Argon2PasswordHasher.

Las contraseñas no se almacenan en texto plano.

8.4.4. Autenticación JWT

La emisión y validación de tokens utiliza PyJWT.

El adaptador JWTAccessTokens encapsula la generación y validación de los tokens utilizados por el backend.

La configuración incluye JWT_SECRET_KEY y ACCESS_TOKEN_EXPIRE_MINUTES.

El tiempo predeterminado documentado para los tokens de acceso es de 60 minutos.

La solución utiliza autenticación basada en tokens de acceso. No se incluye un mecanismo de refresh tokens como funcionalidad implementada.

8.4.5. Protección de endpoints

El componente get_current_user verifica las credenciales proporcionadas mediante el mecanismo HTTP Bearer.

Su utilización permite identificar al usuario y restringir el acceso a las operaciones que requieren autenticación.

Los resultados persistentes se asocian con el identificador del usuario correspondiente.

8.5. Base de datos relacional PostgreSQL

PULSO utiliza PostgreSQL 16 para almacenar cuentas y resultados de análisis.

La base se configura mediante DATABASE_URL y utiliza SQLAlchemy y psycopg para el acceso desde el backend.

8.5.1. Tabla de usuarios

La migración 001_create_users.sql establece la tabla users.

Columna	Tipo	Restricción
id	UUID	Clave primaria
email	VARCHAR(254)	Único y no nulo
password_hash	VARCHAR(255)	No nulo
created_at	TIMESTAMPTZ	Fecha de creación

Esta estructura conserva los datos necesarios para identificar cuentas y verificar credenciales.

8.5.2. Tabla de análisis

La migración 002_create_analysis_history.sql establece la tabla analyses.

Columna	Tipo	Restricción
id	UUID	Clave primaria
user_id	UUID	Clave foránea
probability	DOUBLE PRECISION	Entre 0 y 1
classification	SMALLINT	Valor binario
threshold	DOUBLE PRECISION	Entre 0 y 1
model_version	VARCHAR(50)	No nulo
created_at	TIMESTAMPTZ	Fecha de creación

La relación entre users y analyses es de uno a muchos.

Se utiliza el índice ix_analyses_user_created_at para optimizar las consultas de resultados por usuario y fecha.

8.5.3. Repositorios de persistencia

Los adaptadores principales son:

- SQLAlchemyUserRepository: creación y consulta de usuarios.
- SQLAlchemyAnalysisHistoryRepository: almacenamiento y recuperación del historial.

Estos componentes implementan contratos del dominio y encapsulan las operaciones de SQLAlchemy.

8.5.4. Migraciones

El repositorio contiene migraciones SQL versionadas para usuarios e historiales.

Docker Compose permite inicializar una base de datos nueva mediante los scripts montados en el directorio de inicialización de PostgreSQL.

Los cambios posteriores sobre una base inicializada requieren una estrategia de aplicación de migraciones que preserve los datos existentes.

8.6. Historial persistente de análisis

El sistema permite que los usuarios autenticados consulten sus resultados anteriores.

El caso de uso ListAnalysisHistory recupera los registros correspondientes al usuario identificado mediante JWT.

La consulta se realiza mediante GET /api/analyses, utilizando paginación.

Parámetro	Valor predeterminado	Restricción
limit	20	Entre 1 y 100
offset	0	Mayor o igual a 0

El frontend dispone de HistoryPage, que presenta los registros y permite consultar resultados adicionales.

El historial almacena los resultados técnicos de cada análisis, sin conservar permanentemente el perfil cardiovascular completo ni la explicación SHAP.

8.7. Almacenamiento temporal de análisis

Además de la persistencia PostgreSQL, PULSO utiliza InMemoryAnalysisRepository.

Este repositorio conserva temporalmente el contexto completo necesario para solicitar recomendaciones educativas.

Parámetro	Configuración
TTL	900 segundos
Capacidad máxima	100 entradas
Almacenamiento	Memoria del proceso
Persistencia permanente	No
Compartido entre procesos	No

La separación entre ambos mecanismos responde a responsabilidades distintas.

PostgreSQL conserva resultados históricos asociados con usuarios, mientras que el repositorio temporal permite reutilizar perfiles, predicciones y explicaciones durante el proceso RAG.

8.8. Frontend React

El frontend utiliza React, TypeScript y Vite.

La interfaz incorpora las siguientes vistas y funcionalidades:

- AuthPage: registro e inicio de sesión.
- EvaluationPage: formulario de evaluación.
- ResultPage: presentación de predicción y explicación.
- SimulationPage: simulación de escenarios.
- HistoryPage: consulta del historial.
- Header: navegación y acciones de autenticación.

La aplicación gestiona el estado de sesión y utiliza tokens de acceso para comunicarse con los endpoints protegidos.

El cierre de sesión debe limpiar las credenciales y la información de formulario y análisis asociada con la sesión anterior.

La interfaz presenta comportamiento adaptable a diferentes anchos de pantalla. La validación funcional completa en dispositivos móviles requiere comprobaciones específicas.

8.9. Servicio de Machine Learning

ML Service constituye un servicio independiente implementado mediante Python y FastAPI.

Utiliza XGBoost para clasificación binaria y SHAP para explicabilidad.

Las operaciones principales incluyen predicción, explicación individual y simulación.

El modelo PULSO v1 utiliza un umbral de clasificación de 0.38, seleccionado para priorizar la sensibilidad del clasificador.

La comunicación entre backend y ML Service se realiza mediante HTTP/JSON.

8.10. Sistema RAG y persistencia vectorial

El sistema RAG utiliza workflows n8n para generar recomendaciones educativas fundamentadas en documentos seleccionados.

La base vectorial utiliza Supabase, PostgreSQL y pgvector.

La tabla operativa de producción es public.documents_pulso_test, cuya recuperación se realiza mediante public.match_documents_pulso_test.

El corpus comprende 15 documentos normalizados y 160 fragmentos vectorizados con models/gemini-embedding-001, de 3072 dimensiones.

La infraestructura incorpora tres mecanismos de almacenamiento diferenciados:

Mecanismo	Finalidad
PostgreSQL del backend	Persistencia de usuarios e historiales
Supabase/pgvector	Almacenamiento del corpus documental
Memoria del backend	Contextos temporales de análisis

El sistema RAG utiliza predicciones y explicaciones previamente calculadas; no ejecuta nuevamente el modelo cardiovascular para generar recomendaciones.

8.11. Docker Compose e infraestructura

La infraestructura principal se ejecuta mediante Docker Compose.

La configuración contempla cinco servicios:

Servicio	Tecnología	Responsabilidad
postgres	PostgreSQL 16	Usuarios e historiales
ml-service	FastAPI, XGBoost y SHAP	Inferencia y explicabilidad
backend	FastAPI	API, autenticación y casos de uso
frontend	React/Vite	Aplicación web
nginx	NGINX	Proxy inverso

La configuración utiliza healthchecks para comprobar la disponibilidad de PostgreSQL y ML Service.

El backend depende de la disponibilidad de estos servicios para ejecutar las operaciones correspondientes.

PostgreSQL dispone de un volumen persistente para conservar la información entre reinicios de los contenedores.

NGINX proporciona acceso a la aplicación mediante el puerto 80 en la configuración local.

El comando de construcción y ejecución es:

```
docker compose up -d --build
```

El sistema requiere la configuración previa de variables de entorno y la disponibilidad de las integraciones externas utilizadas por RAG.

8.12. Variables de entorno y seguridad

La configuración se gestiona mediante variables de entorno.

Variable	Responsabilidad
POSTGRES_DB	Nombre de la base
POSTGRES_USER	Usuario de PostgreSQL
POSTGRES_PASSWORD	Contraseña de PostgreSQL
DATABASE_URL	Cadena de conexión
JWT_SECRET_KEY	Clave de firma de tokens
ACCESS_TOKEN_EXPIRE_MINUTES	Vigencia de tokens
ML_SERVICE_URL	Dirección del servicio predictivo
ML_TIMEOUT_SECONDS	Tiempo máximo de espera del servicio ML
RAG_SERVICE_URL	Dirección del servicio RAG
RAG_TIMEOUT_SECONDS	Tiempo máximo de espera del RAG
ANALYSIS_CACHE_TTL_SECONDS	Tiempo de retención temporal
ANALYSIS_CACHE_MAX_ENTRIES	Capacidad del repositorio temporal

El archivo .env.example proporciona una referencia de configuración sin publicar credenciales reales.

Las contraseñas, claves JWT y credenciales externas deben mantenerse fuera del código fuente y de los archivos versionados.

8.13. Pruebas automatizadas

El proyecto incorpora pruebas para diferentes componentes y niveles de la arquitectura.

Categoría	Propósito
Dominio	Verificar reglas y objetos
Aplicación	Comprobar casos de uso
Infraestructura	Verificar clientes y repositorios
Interfaces HTTP	Validar endpoints y contratos
Autenticación	Comprobar registro, inicio de sesión y acceso protegido
Historial	Verificar persistencia, asociación y consulta
Arquitectura	Detectar dependencias prohibidas

El archivo backend/tests/test_architecture.py analiza imports para comprobar las restricciones de dependencia entre capas.

Las pruebas de integración con servicios simulados se diferencian de las pruebas que requieren infraestructura real.

Durante la validación de la implementación de autenticación e historial se reportaron los siguientes resultados:

Verificación	Resultado
Pruebas del backend	88 aprobadas (ejecución E7, 13/11/2026)
Pruebas del frontend	24 aprobadas
Comprobación de tipos TypeScript	Correcta
Lint frontend	Correcto
Build de frontend	Correcto

Estos resultados corresponden a las ejecuciones técnicas reportadas y deben asociarse con las evidencias de ejecución pertinentes.

Validación funcional E7 (13 de noviembre de 2026). La ejecución de python -m pytest backend/tests -q finalizó con 88 pruebas aprobadas y dos advertencias de deprecación de dependencias. El archivo backend/tests/interfaces/test_http_schemas.py registró 8 pruebas aprobadas, incluidas regresiones para el cálculo automático del IMC y el rechazo de escenarios simulados con presión sistólica no superior a la diastólica.

En un entorno de integración Docker Compose con PostgreSQL real se verificaron registro e inicio de sesión, JWT, predicción XGBoost v1 y tres factores SHAP, persistencia de análisis e historial por usuario, controles de acceso entre dos cuentas y paginación básica. Una simulación válida produjo resultados consistentes y conservó el análisis original; una simulación inválida devolvió HTTP 422 tras la corrección.

La integración Backend–n8n RAG generó una respuesta con resumen, tres recomendaciones, cinco referencias estructuralmente completas y aviso de carácter educativo. Se verificaron la consistencia de identificadores, rangos y puntuaciones de las fuentes y las respuestas negativas HTTP 404/401 para los escenarios probados. Estas comprobaciones no constituyen una auditoría semántica de la pertinencia documental.

La corrección de los esquemas y de sus pruebas se publicó en la rama Informe_Backend mediante el commit 2a9ca83. Se mantiene como limitación conocida que el ML Service puede emitir HTTP 500 por serialización de un ValueError ante determinadas solicitudes inválidas enviadas directamente al servicio; la API pública probada las rechaza antes con HTTP 422. Los resultados locales no sustituyen la verificación independiente de los checks de GitHub Actions.

8.14. Integración continua

PULSO utiliza GitHub Actions para automatizar verificaciones relacionadas con frontend, backend, Machine Learning e infraestructura.

Los principales procesos de validación comprenden:

- Backend Validation.
- Frontend Validation.
- ML Service Validation.
- ML Training Validation.
- Infrastructure Validation.

Las verificaciones permiten comprobar pruebas automatizadas, análisis estático, compilación y configuraciones de infraestructura.

La configuración de CI contempla las variables necesarias para comprobar el entorno con autenticación JWT.

Las validaciones que emplean servicios simulados permiten verificar contratos y configuraciones de forma controlada, pero no deben confundirse con pruebas de extremo a extremo utilizando todas las integraciones reales.

8.15. Documentación técnica

El repositorio incorpora documentación que describe la organización, ejecución y mantenimiento del sistema.

Entre los documentos de referencia se encuentran:

- README.md.
- docs/ARQUITECTURA_BACKEND.md.
- Documentación del modelo de Machine Learning.
- Documentación de infraestructura y pruebas.
- Informes del proyecto y del corpus RAG.

FastAPI proporciona documentación OpenAPI de los endpoints disponibles.

La documentación técnica permite relacionar las funcionalidades implementadas con sus componentes, contratos y mecanismos de verificación.

8.16. Alcance del módulo de usuarios

La implementación actual incluye registro, inicio de sesión, autenticación JWT y consulta del historial persistente de análisis.

Sin embargo, se diferencian estas funcionalidades de otras capacidades de administración de usuarios no contempladas como implementadas en esta versión, entre ellas:

- Renovación de sesiones mediante refresh tokens.
- Administración de múltiples roles.
- Registro y gestión específica de consentimientos.
- Eliminación de cuentas mediante un caso de uso propio.
- Panel administrativo de permisos.

Estas capacidades constituyen posibles ampliaciones y no deben confundirse con las funcionalidades de autenticación y persistencia existentes.

8.17. Conclusión

La base técnica de PULSO integra una aplicación web React, un backend FastAPI organizado mediante Clean Architecture adaptada, un servicio independiente de Machine Learning, una base de datos PostgreSQL para cuentas e historiales y un sistema RAG conectado con Supabase.

La arquitectura incorpora mecanismos de autenticación JWT, almacenamiento seguro de contraseñas mediante Argon2, protección de endpoints y acceso a resultados persistentes asociados con cada usuario.

El almacenamiento temporal de análisis se mantiene separado de la persistencia relacional y del corpus vectorial, permitiendo conservar responsabilidades diferenciadas.

Docker Compose coordina los cinco servicios principales, mientras que las herramientas de pruebas automatizadas y GitHub Actions proporcionan mecanismos para verificar la calidad y consistencia de la implementación.

En conjunto, la base técnica ofrece una estructura modular, verificable y preparada para continuar la evolución funcional de PULSO, manteniendo la separación entre sus componentes predictivos, educativos, de persistencia y de seguridad.